

[↑ 分享](#) ...

Brelliothe

MA-FSTSP

你能否在github上看到这个仓库？给我返回其中的一些代码片段证明你看到了

我会先确认 GitHub 上是否存在 Brelliothe/MA-FSTSP，再打开仓库里的源文件，用少量原始代码片段作为证据。

Thought for 32s >

能看到。准确说，我能通过公开 GitHub 页面看到 Brelliothe/MA-FSTSP，页面标注为 **Public**，仓库名为 Brelliothe / MA-FSTSP；根目录里有 `src/`、`README.md`、`experiments.py`、`plot.py`、`problem.py`、`requirements.txt`、`utils.py` 等文件。

GitHub

下面是几个代码片段作为证据。

`src/fstsp.py` 中定义了核心类 `MultiAgentFlyingSidekickTSP`： [GitHub](#)

Python

运行

```
import elkai
import gurobipy as gp
from gurobipy import GRB
import math
import networkx as nx
import numpy as np
from baseline import Baseline
from utils import mst_partition

class MultiAgentFlyingSidekickTSP(Baseline):
    def __init__(self, graph, depots, cities, distance, drone,
                  limit=1.5, speed=1.6, theta=(0.5, 0.5)):
        super().__init__(graph, depots, cities, distance, drone, limit, speed)
        self.groups = {depot: [] for depot in depots}
        self.solution = []
        self.cost = 0
```

同一文件里还有基于 Gurobi 的 Set-TSP 建模片段：[GitHub](#)

Python

运行

```
model = gp.Model('Set-TSP')
model.setParam("OutputFlag", 0)

select = model.addMVar((n, n), vtype=GRB.BINARY)
model.addConstrs(select[u, u] == 0 for u in range(n))
model.addConstrs(np.ones((n,)) @ select[:, v] == 1 for v in range(n))
model.addConstrs(np.ones((n,)) @ select[u, :] == 1 for u in range(n))
```

src/baseline.py 中定义了基类模板：[GitHub](#)

Python

运行

```
# the file defines the template of the baseline class

class Baseline:
    def __init__(self, graph, depots, cities, distance, drone, limit, speed):
        self.graph = graph
        self.depots = depots
        self.cities = cities
        self.distance = distance
        self.name = name
        self.drone = drone
        self.limit = limit
        self.speed = speed
```

problem.py 中有生成/读取路网实例的函数，例如 manhattan() 和 cambridge()：[GitHub](#)

Python

运行

```
def manhattan():
    """read the manhattan road map from file and return it as a networkx graph"""
    g = nx.MultiDiGraph(nx.read_graphml('nyc.graphml'))
    manhattan_graph = nx.MultiDiGraph()
```

结论：可以看到这个仓库；它是公开仓库，且 README 称其为论文 “Optimization of Multi-Agent Flying Sidekick Traveling Salesman Problem over Road Networks” 的官方实现。[GitHub](#)

     ...  来源

我现在想学习这个仓库，假设我是一个python纯新手，给我讲一下这个仓库中的类、数据结构、函数、主要逻辑、函数调用关系等相关内容

我会按“新手能读懂源码”的方式拆解：先确认仓库文件结构和关键源码，再把类、数据结构、函数职责、主流程和调用链串起来。不会假设你已经懂 Python 或优化建模。

目前我已经确认 src 目录下实际只有 baseline.py、fstsp.py、hc_vns.py、lp.py、lrm.py 这几个算法文件；仓库根目录还有实验、画图、问题构造和工具函数文件。接下来我会把它们按“入口—问题数据—算法类—输出结果”的顺序讲。

Received app response ▾

有一个对新手很重要的细节：这个仓库不是“教学型代码”，而是论文复现实验代码，注释较少，而且 baseline.py 里有一个明显的源码问题：self.name = name 中的 name 没有定义。也就是说，若不修这一行，直接实例化算法类很可能会报错。

Thought for 2m 5s ▸

下面按“Python 纯新手读源码”的方式讲这个仓库。核心结论先说：这个仓库是论文 **Multi-Agent Flying Sidekick TSP over Road Networks** 的实现，主要做的是“多辆卡车 + 多架无人机 + 多个仓库/车场 + 多个客户点”的协同配送路径优化。README 说明它的算法主要在 src/fstsp.py，实验入口是 experiments.py，画图入口是 plot.py。 [GitHub](#)

1. 这个仓库的总体结构

仓库根目录里主要有：

```

MA-FSTSP/
├─ experiments.py      # 实验入口：跑论文中的实验
├─ plot.py             # 画图
├─ problem.py          # 构造问题实例：地图、仓库、客户、距离矩阵
├─ utils.py            # 工具函数：距离计算、MST 分组、TSP 辅助等
├─ requirements.txt    # 依赖包
└─ src/
    ├─ baseline.py     # 所有算法类的父类模板
    ├─ fstsp.py        # 作者主算法：MultiAgentFlyingSidekickTSP
    ├─ hc_vns.py        # Hill Climbing + VNS 启发式基线
    ├─ lp.py            # 线性规划 / 整数规划基线
    └─ lrm.py          # Linear Relaxed Master Problem 基线
    
```

src 目录下实际列出的算法文件就是 baseline.py 、 fstsp.py 、 hc_vns.py 、 lp.py 、 lrm.py 。

GitHub

2. 先理解这个仓库在解决什么问题

可以把问题理解成：

有多个 depot，也就是多个卡车出发点。每个 depot 对应一辆卡车，卡车上带若干架无人机。客户点需要被服务。卡车只能沿道路网络走，无人机可以按直线距离飞。目标是让所有客户被服务，同时总耗时或总代价尽量小。

代码里常见变量含义如下：

变量	新手解释	在问题里的含义
graph	图结构	道路网络，节点是路口/道路点，边是道路
depots	列表/数组	仓库、车场、卡车起点
cities	列表/数组	客户点
distance	字典	记录卡车距离和无人机距离
drone	整数	每辆卡车携带的无人机数量
limit	数值	无人机最大续航距离
speed	数值	无人机相对卡车的速度系数

变量	新手解释	在问题里的含义
solution	字典	输出路径
cost	数值	路径总代价/总时间

problem.py 中构造的 distance 是一个双层结构：

Python

运行

```
distance = {
    'truck': ...,
    'drone': ...
}
```

其中 truck 是道路最短路距离， drone 是两点之间的直线/球面距离。代码中用 `nx.all_pairs_dijkstra_path_length` 计算卡车道路距离，用 `haversine` 计算无人机距离。

problem

3. Python 新手要先看懂的几个基本概念

3.1 class 是“模板”

例如：

Python

运行

```
class MultiAgentFlyingSidekickTSP(Baseline):
```

意思是定义一个类，叫 `MultiAgentFlyingSidekickTSP` 。它继承了 `Baseline` 。你可以把类理解成“算法模板”，每次调用：

Python

运行

```
model = MultiAgentFlyingSidekickTSP(...)
```

就是创建一个算法对象。

3.2 self.xxx 是对象自己的变量

例如 `fstsp.py` 中：

Python

运行

```
self.groups = {depot: [] for depot in depots}
self.solution = []
self.cost = 0
```

意思是这个算法对象内部保存了：

- groups：每个 depot 分配到哪些客户；
- solution：最终路径；
- cost：最终代价。

这些初始化逻辑在 `MultiAgentFlyingSidekickTSP.__init__()` 中。 [GitHub](#)

3.3 def solve(self) 是算法入口

几乎每个算法类都有 `solve()`。你只需要记住：

Python

运行

```
solution, cost = model.solve()
```

就是“运行这个算法，得到路径和代价”。

4. Baseline：所有算法的父类模板

`src/baseline.py` 定义了最基础的父类：

Python

运行

```
class Baseline:
    def __init__(self, graph, depots, cities, distance, drone, limit, speed):
        self.graph = graph
        self.depots = depots
        self.cities = cities
        self.distance = distance
        self.name = name
        self.drone = drone
        self.limit = limit
        self.speed = speed
```

它的作用是统一保存公共数据：地图、仓库、客户、距离、无人机数量、续航、速度等。它

还规定了解的输出格式：

Python

运行

```
{'truck': [node1, node2, ...],
 'drone': [[[node1, city, node2], ...], ...]}
```

也就是说：

- truck 是卡车经过的节点序列；
- drone 是无人机任务列表；
- 每个无人机任务 [node1, city, node2] 表示：无人机从 node1 起飞，服务 city，在 node2 降落。

baseline

不过这里有一个**明显源码问题**：self.name = name 中的 name 没有作为参数传入，也没有在函数内部定义。按 Python 语义，这会导致 NameError。学习或运行前，建议先改成：

Python

运行

```
self.name = None
```

或者直接删除这一行。

baseline

5. problem.py：负责构造问题数据

这个文件不是算法，而是“造题器”。

主要函数有：

5.1 manhattan()

读取 nyc.graphml，构造曼哈顿道路网络。

它会：

1. 用 NetworkX 读取图；
2. 给每个节点保存经纬度 pos；
3. 给每条边计算道路距离；
4. 如果没有缓存文件，就计算所有节点两两之间的最短路距离。

problem

5.2 cambridge()

从 OSMnx 下载/构造 Boston, MA 的道路网络，然后取最大强连通子图。 problem

5.3 small_instance(...)

从曼哈顿图中截取一个小子图，用于小规模实验。

它返回：

Python

运行

```
subgraph, _depots, _cities, distance
```

其中：

- subgraph : 小道路图;
- _depots : 多组随机 depot;
- _cities : 多组随机 customer;
- distance : 卡车/无人机距离矩阵。 problem

5.4 multiagent_instance_on_manhattan(...)

在完整曼哈顿图上随机抽 depot 和 city。

5.5 multiagent_instance_on_cambridge(...)

在 Cambridge/Boston 图上随机抽 depot 和 city，并用 pickle 缓存距离矩阵。 problem

6. utils.py : 工具函数

这个文件放的是通用辅助逻辑。

最重要的函数有：

6.1 haversine(pos1, pos2)

计算地球表面两点之间的近似距离。无人机距离主要靠它算。 utils

6.2 nearest_node(graph, location)

给一个坐标，找图中最近的节点。

6.3 nearest_node_except_self(graph, name)

找某个节点最近的邻居，常用于把客户点映射到附近的道路节点。

utils

6.4 mst_partition(graph, depots, cities)

这是一个关键函数。作用是：

根据一个完全图的最小生成树，把客户分配给不同 depot。

输出格式大概是：

Python

运行

```
{
    depot1: [cityA, cityB],
    depot2: [cityC, cityD]
}
```

作者主算法 fstsp.py 会调用它来决定“哪个 depot 负责哪些客户”。

utils

6.5 asymmetric_traveling_salesman_problem(...)

把非对称 TSP 做一个图变换，然后调用 NetworkX 的近似算法求路径。这个主要给

lrm.py 用。

utils

7. 主算法类： MultiAgentFlyingSidekickTSP

这是最值得重点学习的类，位于 src/fstsp.py 。 README 也明确说作者算法实现就在这个文件里。

GitHub

类名：

Python

运行

```
class MultiAgentFlyingSidekickTSP(Baseline):
```

简称可以叫 MA-FSTSP 主算法类。

8. MultiAgentFlyingSidekickTSP 的核心数据结构

8.1 self.groups

Python

运行

```
self.groups = {depot: [] for depot in depots}
```

表示每个 depot 分配到的客户列表。

例如：

Python

运行

```
{
    10: [3, 7, 9],
    25: [4, 8]
}
```

意思是 depot 10 服务客户 3、7、9；depot 25 服务客户 4、8。 GitHub

8.2 self.regions

Python

运行

```
self.regions = {
    city: [node for node in self.graph.nodes
           if self.distance['drone'][node][city] < self.limit / 2]
    for city in cities
}
```

这个很重要。

它表示：对于每个客户 city，哪些道路节点 node 离它足够近，可以作为无人机起飞/降落节点。

直观理解：

客户附近有一圈“可服务区域”。卡车如果经过这个区域边界附近，无人机就可以飞出去服务客户，再飞回来。

regions 在后续动态规划中大量使用。 GitHub

8.3 convex_sets

虽然代码叫 convex_sets，作者自己也写了注释：

Python

运行

```
# not necessary a convex set, a wrong name
```

也就是说这个名字不严谨。它实际是“每个客户对应的一组候选道路节点”。

`get_convex_sets()` 会为每个客户找一批距离足够近的节点,
`get_boundary_convex_sets()` 再找这些区域的边界节点。 [GitHub](#)

8.4 solution

统一格式是：

Python

运行

```
{
    'truck': [depot, node1, node2, depot],
    'drone': [
        [[takeoff, city, landing], ...],
        ...
    ]
}
```

主算法中 `convert()` 会把内部解转换成这个格式。 [GitHub](#)

9. 主算法的 `solve()` 调用关系

`fstsp.py` 中主入口是：

Python

运行

```
def solve(self):
    convex_sets = self.get_boundary_convex_sets(self.theta[0])
    self.set_mst(convex_sets)
    for depot in self.depots:
        convex_set = [[depot]] + [convex_sets[city] for city in self.groups[depot]]
        solution = self.single_solution(depot, convex_set)
        self.solution.append(self.convert(solution))
    return self.solution, self.cost
```

调用链可以画成：

```

experiments.py
↓
model = MultiAgentFlyingSidekickTSP(...)
↓
model.solve()
↓
get_boundary_convex_sets()
↓
get_convex_sets()
↓
set_mst()
↓
utils.mst_partition()
↓
对每个 depot:
    single_solution()
    ↓
    get_seq()
    ↓
    如果 theta[1] == 0: lkh()
    否则: set_tsp()
    ↓
    local_search_multi_drone_appr()
    ↓
    convert()
↓
返回 solution, cost

```

这就是主算法最核心的执行流程。 GitHub

10. 主算法每一步具体在干什么

10.1 第一步：找客户的候选服务区域

Python

运行

```
convex_sets = self.get_boundary_convex_sets(self.theta[0])
```

它先调用 `get_convex_sets(theta)`，找到每个客户附近可作为无人机起降点的道路节点；
然后调用 `get_boundary_convex_sets(theta)`，只保留边界节点。 GitHub

直观解释：

不需要枚举地图上所有道路节点，只考虑客户附近的一小批候选起降点，降低计算规模。

10.2 第二步：用 MST 给客户分组

Python

运行

```
self.set_mst(convex_sets)
```

`set_mst()` 会构造一个完全图，图中的节点包括：

- depot;
- city。

边权表示 depot-city 或 city-city 之间的近似服务代价。然后调用：

Python

运行

```
self.groups = mst_partition(graph, self.depots, self.cities)
```

把客户分配到不同 depot。 [GitHub](#)

直观解释：

先决定“每辆卡车负责哪些客户”，再分别求每辆卡车的路径。

10.3 第三步：对每个 depot 单独求解

Python

运行

```
for depot in self.depots:
    convex_set = [[depot]] + [convex_sets[city] for city in self.groups[depot]]
    solution = self.single_solution(depot, convex_set)
```

如果一个 depot 没有客户，直接返回：

Python

运行

```
{'truck': [depot, depot], 'drone': []}
```

否则进入 `single_solution()` 。 [GitHub](#)

10.4 第四步：确定客户访问顺序 `get_seq()`

Python

运行

```
seq = self.get_seq(depot, convex_sets)
```

这里有两个分支：

情况 A: `theta[1] == 0`

用 `lkh()` 求普通 TSP 访问顺序：

Python

运行

```
route = elkai.DistanceMatrix(int_matrix).solve_tsp()
```

也就是先不考虑复杂的集合访问，只用卡车距离求一个 TSP 顺序。 [GitHub](#)

情况 B: `theta[1] != 0`

调用 `set_tsp()`，用 Gurobi 建立一个 Set-TSP 模型。

它会创建：

Python

运行

```
select = model.addMVar((n, n), vtype=GRB.BINARY)
flow = model.addMVar((n, n), vtype=GRB.CONTINUOUS)
```

其中：

- `select[u, v] = 1` 表示从集合 `u` 访问到集合 `v`；
- `flow` 用来消除子回路；
- `internal` 表示集合内部选哪个起降点；
- `external` 表示集合之间选哪两个道路节点连接。 [GitHub](#)

直观解释：

它不是单纯决定“客户访问顺序”，还要考虑每个客户附近选哪个道路节点作为服务节点。

10.5 第五步：局部搜索 / 动态规划安排卡车和无人机

最复杂的函数是：

Python

运行

```
local_search_multi_drone_appr(self, seq, depot)
```

它用到了几个动态规划表：

Python

运行

```
value
appr
tour
group
prefix
```

含义可以这样理解：

名称	作用
value	当前走到某个阶段、某个节点的最小代价
appr	对一段无人机服务组合的近似耗时
tour	对应的局部路径记录
group	当前最多能把几个客户合并成一组由无人机服务
prefix	到当前状态为止的完整路径前缀

代码会比较两种服务方式：

- 1. 客户由卡车直接访问；
- 2. 客户由无人机从某个节点起飞服务，再在某个节点降落。

如果无人机飞行距离超过 limit，就不能用无人机服务：

Python

运行

```
if consumption > self.limit:
    continue
```

最终返回：

Python

运行

```
return prefix[...] [depot], value[...] [depot]
```

也就是一条具体路径和对应代价。 GitHub

11. hc_vns.py：Hill Climbing + Variable Neighborhood Search

这个是启发式基线算法。

类名：

Python

运行

```
class HillClimbingVariableNeighborhoodSearch(Baseline):
```

它的主流程是：

```
solve()
↓
partition()
↓
对每个 depot:
    single_solution()
    ↓
    init_solution()
    ↓
    重复 rounds 次:
        neighborhood_search()
    ↓
    solution_cost()
    ↓
    convert()
```

11.1 partition()

把每个客户分配给最近的 depot，最近的标准是道路最短路距离。 hc_vns

11.2 init_solution(nodes)

先构造一个普通 TSP 初始解。注意它一开始默认所有客户都由卡车访问：

Python

运行

```
solution = [(i, -1, i) for i in solution]
```

其中 -1 表示“卡车服务”，非负数表示“第几架无人机服务”。 hc_vns

11.3 neighborhood_search(solution)

随机选择一种邻域操作：

1. 改变无人机起降位置；
2. 把一个原本由卡车访问的客户改成无人机访问；
3. 交换两个访问节点的顺序。

这是典型的局部搜索思想：每次在当前解附近尝试小改动，如果更好就接受。 hc_vns

11.4 solution_cost(solution)

计算整条路径的耗时。关键是卡车和无人机并行时，要取最大值：

Python

运行

```
cost[i] = max(cost[i], cost[j] + drone_distance / self.speed)
```

意思是：卡车到达降落点后，如果无人机还没回来，卡车要等无人机。 hc_vns

12. lp.py：整数规划基线

类名：

Python

运行

```
class LinearProgramming(Baseline):
```

它不是主要算法，更像是一个小规模精确/近似基线。

流程：

```

solve()
↓
induce()
↓
构造 Gurobi 模型
↓
添加 city_route、flow、route、goods 等变量
↓
最小化路径代价
↓
返回 cost

```

12.1 induce()

先构造一个子图 `subgraph` 。它只保留：

- depot;
- city;
- 每个 city 可由无人机覆盖区域的边界节点。

这样可以缩小优化模型规模。 lp

12.2 solve()

里面建立 Gurobi 模型：

Python

运行

```

city_route = model.addMVar((num_locs, num_locs), vtype=GRB.BINARY)
flow = model.addMVar((num_locs, num_locs), vtype=GRB.CONTINUOUS)
route = model.addMVar((num_nodes, num_nodes), vtype=GRB.BINARY)
goods = model.addMVar((num_nodes, num_nodes), vtype=GRB.CONTINUOUS)

```

其中：

- `city_route` ：客户/depot 层面的访问关系；
- `flow` ：防止子回路；
- `route` ：实际道路节点之间的路径；
- `goods` ：保证路径连通到 depot。 lp

这个类基本只更新 `self.cost` ，注释也写了“no need to return the solution”。 lp

13. lrm.py : Linear Relaxed Master Problem

这是另一个基线，思路更复杂，接近列生成/主问题-定价问题框架。

类名：

Python

运行

```
class LinearRelaxedMasterProblem(Baseline):
```

代码注释说它来自 **Scheduling trucks and drones for cooperative deliveries**，并实现了 VNS 加速策略。

它的主要数据结构有：

变量	含义
routes	当前已有的候选路线集合
alpha[i][j][k]	卡车 k 是否从节点 i 到节点 j
beta[i][j][k][d]	卡车 k 上无人机 d 是否从 i 飞到 j
epsilon[k]	卡车 k 是否被使用
y[i][j][d]	路线中无人机服务记录
m	Big-M 模型中的大常数

主流程：

```

solve()
↓
initial_solution()
↓
最多循环 5 次:
    master_problem()
    pricing_problem()
    ↓
    solve_truck_route()
    solve_drone_route()
    neighborhood_search()
    shake()
↓
get_solution()
↓
solve_optimal_pp()
↓
返回 solution, cost

```

`solve()` 的实际代码最后是:

Python

运行

```

self.initial_solution()
count, loop = 0, True
while loop and count < 5:
    count += 1
    routes, theta, _ = self.master_problem()
    for route in routes:
        loop = loop and (self.pricing_problem(route, theta) > 0)
self.get_solution()
return self.solution, self.cost

```

GitHub

新手可以先不深究这个类, 因为它涉及 Gurobi、Big-M、对偶变量、主问题、定价问题, 门槛比 `fstsp.py` 高很多。

14. experiments.py : 实验入口

这个文件告诉你仓库作者是怎么调用算法的。

例如小规模实验 `test_small_instance()` 中, 它会依次创建四种模型:

Python

运行

```
LinearRelaxedMasterProblem(...)
HillClimbingVariableNeighborhoodSearch(...)
MultiAgentFlyingSidekickTSP(...)
LinearProgramming(...)
```

每个模型都调用:

Python

运行

```
solution, cost = model.solve()
```

然后统计平均 `cost` 和平均运行时间。 `experiments`

所以你要理解整个仓库, 只需要抓住这个模式:

Python

运行

```
graph, depots, cities, distance = 生成问题实例
model = 某个算法类(graph, depots, cities, distance, drone)
solution, cost = model.solve()
```

15. 全仓库主调用关系图

可以把整个仓库理解成下面这样:

```

experiments.py
├── problem.py
│   ├── manhattan()
│   ├── cambridge()
│   ├── small_instance()
│   ├── multiagent_instance_on_manhattan()
│   └── multiagent_instance_on_cambridge()
├── src/fstsp.py
│   └── MultiAgentFlyingSidekickTSP.solve()
├── src/hc_vns.py
│   └── HillClimbingVariableNeighborhoodSearch.solve()
├── src/lp.py
│   └── LinearProgramming.solve()
└── src/lrmp.py
    └── LinearRelaxedMasterProblem.solve()

```

辅助函数关系：

```

problem.py
└── utils.haversine()

fstsp.py
└── utils.mst_partition()

lrmp.py
├── utils.nearest_node_except_self()
└── utils.asymmetric_traveling_salesman_problem()

```

16. 新手应该按什么顺序读

建议按这个顺序：

第一遍：只看运行入口

先读：

```
experiments.py
```

重点看：

Python

运行

```
model = MultiAgentFlyingSidekickTSP(...)
solution, cost = model.solve()
```

你会知道“模型怎么被调用”。

第二遍：看问题数据怎么生成

读：

```
problem.py
utils.py
```

重点理解：

Python

运行

```
graph
depots
cities
distance
```

特别是 `distance['truck']` 和 `distance['drone']` 的区别。

第三遍：读 Baseline

读：

```
src/baseline.py
```

重点理解统一输出格式：

Python

运行

```
{'truck': [...], 'drone': [...]}
```

同时修掉：

Python

运行

```
self.name = name
```

这个问题。

第四遍：读主算法 fstsp.py

重点只看这些函数：

```
__init__()
solve()
get_convex_sets()
get_boundary_convex_sets()
set_mst()
single_solution()
get_seq()
set_tsp()
local_search_multi_drone_appr()
convert()
```

先不要一上来就钻 `local_search_multi_drone_appr()`，它是全仓库最难的函数之一。

第五遍：读基线算法

读：

```
hc_vns.py
lp.py
lrm.py
```

优先级：

```
hc_vns.py > lp.py > lrm.py
```

因为 `hc_vns.py` 最直观，`lrm.py` 最难。

17. 这个仓库运行前的现实问题

实话说，这个仓库对 Python 新手并不友好，主要有几个坑：

1. `baseline.py` 里 `self.name = name` 未定义，直接运行可能报错。 baseline
2. 依赖里有 `gurobipy==11.0.0`，这通常需要 Gurobi 环境和 license；否则 `lp.py`、`lrmp.py`、`fstsp.py` 中涉及 Gurobi 的部分无法正常跑。 requirements
3. README 的 clone 命令写的是 `MixTSP`，但当前仓库名是 `MA-FSTSP`，这看起来是 README 没同步更新。 GitHub
4. `manhattan()` 依赖 `nyc.graphml`，如果本地没有这个文件，曼哈顿实验跑不起来。 problem
5. 代码偏论文实验复现，不是工程化项目，缺少单元测试、配置文件和详细文档。

18. 最重要的一句话总结

这个仓库的核心逻辑是：

`problem.py` 生成道路网络和距离矩阵；`experiments.py` 创建算法对象；每个算法类都继承 `Baseline` 并实现 `solve()`；主算法 `MultiAgentFlyingSidekickTSP` 先把客户分配给 depot，再为每个 depot 求客户访问顺序，最后用动态规划/局部搜索安排卡车和无人机协同服务，输出统一格式的 `solution` 和 `cost`。

     ...  来源